

PowerShell Incident Response Cheat Sheet

When every second counts, efficient evidence collection makes all the difference. This quick reference is designed to help analysts rapidly collect and triage key artifacts during a live response investigation.

Core Cmdlets

A *cmdlet* is a lightweight, single-function command built into PowerShell. Cmdlets are designed to perform specific tasks, like retrieving system information or managing processes. They're the building blocks for gathering and analyzing data efficiently during investigations.

Command	Alias	Description
<code>Get-Item</code>	<code>gi</code>	Retrieves metadata about a single file or registry key.
<code>Get-ChildItem</code>	<code>dir</code> , <code>gci</code> , <code>ls</code>	Lists files and directories (used for browsing file system paths and artifacts).
<code>Get-ItemProperty</code>	<code>gp</code>	Retrieves file or registry key properties (LastAccessTime, registry value name/data, etc.).
<code>Get-Content</code>	<code>cat</code> , <code>gc</code> , <code>type</code>	Reads contents of text-based log files, scripts, and configuration files.
<code>Select-String</code>	<code>sls</code>	Searches file contents or command output for keywords.
<code>Get-WmiObject</code> (<i>deprecated</i>)	<code>gwmi</code>	Queries WMI providers for system information (legacy use).
<code>Get-CimInstance</code>	<code>gcim</code>	Queries CIM/WMI providers for system info (modern replacement for <code>Get-WmiObject</code>).
<code>Get-Process</code>	<code>gps</code> , <code>ps</code>	Lists all running processes with names, PIDs, and resource usage.
<code>Get-Service</code>	<code>gsv</code>	Lists services and their states (running, stopped, startup type, etc.).
<code>Get-ScheduledTask</code>	(<i>none</i>)	Lists scheduled tasks and their execution details.
<code>Get-WinEvent</code>	(<i>none</i>)	Retrieves event logs (System, Security, Application, etc.) with a number of filtering options.

<code>Get-NetTCPConnection</code>	<i>(none)</i>	Lists active TCP connections and listening ports, including owning process IDs.
<code>Get-NetAdapter</code>	<i>(none)</i>	Displays network interface details and configuration.
<code>Get-NetIPAddress</code>	<i>(none)</i>	Display IP address configuration (IPv4/IPv6) and IP interfaces with which addresses are associated.
<code>Get-SmbShare</code>	<i>(none)</i>	Lists SMB file shares hosted on the system, including their paths.
<code>Get-DnsClientCache</code>	<i>(none)</i>	Shows cached DNS query results on the local machine.
<code>Get-LocalUser</code>	<code>glu</code>	Lists local user accounts on the system.
<code>Get-LocalGroupMember</code>	<code>glgm</code>	Shows members of local groups (e.g., Administrators).
<code>Get-HotFix</code>	<i>(none)</i>	Lists installed updates and patches on the system.
<code>Get-PnpDevice</code>	<i>(none)</i>	Enumerates plug-and-play devices, including USB hardware history.
<code>Get-MpPreference</code>	<i>(none)</i>	Retrieves Windows Defender preferences, exclusions, and scan settings.
<code>Get-Acl</code>	<i>(none)</i>	Displays file or registry permissions.
<code>Get-Date</code>	<i>(none)</i>	Displays current system time.
<code>Get-FileHash</code>	<i>(none)</i>	Generates file hashes (MD5/SHA1/SHA256) for integrity checks or IOC matching.
<code>Write-Host</code>	<i>(none)</i>	Displays messages directly to the console. Commonly used for interactive output within scripts.

Formatting and Output Cmdlets

During investigations, we often need to sort, filter, and transform that information to surface patterns and find meaning. PowerShell's formatting cmdlets let you quickly transform raw output into something clear and actionable.

Command	Alias	Description
<code>Format-Table</code>	<code>ft</code>	Displays output in a table format with columns.
<code>Format-List</code>	<code>fl</code>	Displays properties in a list format.
<code>Format-Wide</code>	<code>fw</code>	Shows only a single property per object, displayed in multiple columns across the console width.
<code>Select-Object</code>	<code>select</code>	Chooses specific properties from objects and can select the first or last <i>N</i> results.
<code>Sort-Object</code>	<code>sort</code>	Sorts objects in ascending or descending order based on one or more properties.
<code>Group-Object</code>	<code>group</code>	Groups objects that share a common property and displays a count of each group.
<code>Measure-Object</code>	<code>measure</code>	Calculates statistics (<i>count</i> , <i>sum</i> , <i>average</i> , <i>min</i> , <i>max</i>) for numeric or string properties of objects.
<code>Out-String</code>	<i>(none)</i>	Converts objects into a string representation.
<code>Out-File</code>	<i>(none)</i>	Sends formatted output to a text file instead of the console.
<code>ConvertTo-Json</code>	<i>(none)</i>	Converts PowerShell objects into JSON format.
<code>ConvertTo-Csv</code>	<i>(none)</i>	Converts objects into comma-separated values.
<code>Export-Csv</code>	<code>epcsv</code>	Writes object data to a CSV file.

Collection Examples

Below is a **non-exhaustive** set of commands to gather key system information during your investigation. Use these as building blocks to tailor your data collection. In PowerShell, there are often multiple ways to retrieve the same information. It is generally recommended to use more than one method to validate findings and reduce the risk of missing or tampered data.

Remote Collection:

PowerShell allows you to scale your data collection efforts across multiple machines by specifying remote targets.

```
Get-CimInstance -ClassName Win32_OperatingSystem -ComputerName WKS-4F1D, WKS-8N3K
```

The `-ComputerName` parameter specifies the hosts on which you want to run a WMI/CIM operation. This can be a FQDN, a NetBIOS name, or an IP address.

Collect Machine and OS Information:

```
Get-CimInstance -ClassName Win32_OperatingSystem | Select-Object * | Format-List
```

Includes OS version, build, serial number, install date, etc.

```
Get-CimInstance Win32_OperatingSystem | Select-Object LastBootUpTime
```

Retrieves the system's last boot time, showing when the machine was most recently started or restarted.

Collect Local User Accounts and Login Artifacts:

```
Get-LocalUser
```

Lists all local user accounts configured on the machine, including disabled or hidden accounts.

```
Get-CimInstance -ClassName Win32_LoggedOnUser
```

Retrieves information about currently active logon sessions and the users associated with them.

```
Get-CimInstance -ClassName Win32_ComputerSystem | Select-Object Name, UserName
```

Displays the computer's hostname and the username of the currently logged-in user.

Collect Running Process Artifacts:

```
Get-Process
```

Lists all running processes with process name, PID, and resource usage.

```
Get-Process -Name svchost
```

Retrieves a process (or processes) by name (e.g., `svchost`). The `-Name` parameter also supports wildcards (`*`).

```
Get-Process -Id 14012 | Select-Object Name, Path
```

Retrieves a process by its process ID (e.g., `14012`) and lists its full executable path.

```
Get-Process -Name audiodg -Module
```

Displays all DLL modules loaded by the specified process (e.g., `audiodg`), including its name, full path, and size.

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 5 Name, Id, CPU
```

Sorts all processes by CPU usage and returns the top 5 most resource-intensive processes.

```
Get-Process | Where-Object { $_.Path } | ForEach-Object { Get-FileHash $_.Path }
```

Retrieves `SHA256` file hashes for all running processes that have an executable path. The `-Algorithm` argument can be appended to `Get-FileHash` to alternatively specify `SHA1`, or `MD5` hash values.

```
Get-CimInstance -ClassName Win32_Process | Select-Object Name, ProcessId,  
ParentProcessId, ExecutablePath, CommandLine
```

Retrieves running processes with their IDs, parent process IDs, executable paths, and full command lines.

```
Get-CimInstance -ClassName Win32_Process -Filter "Name = 'chrome.exe'"
```

Filters running processes using CIM to show only those with a specific name (e.g., `chrome.exe`).

Collect Network Artifacts:

```
Get-NetTCPConnection -State Established
```

Retrieves information about all TCP connections on the system that are currently in an established state.

```
Get-NetUDPEndpoint | Select-Object LocalAddress, LocalPort, OwningProcess
```

Retrieves all active UDP listening endpoints on the system and displays their local IP address, port number, and owning process ID (PID).

```
Get-NetIPConfiguration; Get-NetIPAddress; Get-NetAdapter; Get-NetRoute
```

Retrieves the system's current IP configuration, network adapter details, and routing table information respectively.

```
Get-DnsClientCache
```

Retrieves the DNS resolver cache entries from the local machine.

```
Get-NetFirewallRule
```

Retrieves all configured Windows Firewall rules, helping to identify active firewall policies and any changes.

Collect Share and Drive Artifacts:

```
Get-SmbShare | ConvertTo-Csv -NoTypeInfo
```

Exports the list of SMB shares on the system into CSV format.

```
Get-SmbConnection
```

Displays active SMB client connections, remote servers, and share names the local system is connected to.

```
Get-PSDrive -PSProvider FileSystem
```

Lists all file system drives, including mapped network drives and local volumes.

```
Get-CimInstance Win32_USBHub
```

Retrieves information about USB hubs and connected USB devices on the system.

```
Get-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Enum\USBSTOR"
```

Reads registry entries that track USB storage devices previously connected to the system.

Collect Autorun Artifacts:

```
Get-ItemProperty -Path "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run"
```

Lists programs set to start automatically at system startup for all users. Change to **HKCU** for the current user.

```
Get-ItemProperty -Path "HKLM:\Software\Microsoft\Windows\CurrentVersion\RunOnce"
```

Lists programs set to start once at system startup for all users. Change to **HKCU** for the current user.

For a full list of registry run keys, see: <https://soc201.com/he1z97>

Collect Service Artifacts:

```
Get-Service | Select-Object Name, DisplayName, Status, StartType
```

Lists all services showing their name, display name, current status (running, stopped), and how they start.

```
Get-Service | Where-Object { $_.Status -eq 'Running' }
```

List only the services that are actively running on the system.

```
Get-Service | Where-Object StartType -eq 'Automatic'
```

Displays services configured to start automatically when the system boots.

```
Get-CimInstance -ClassName Win32_Service | Select-Object Name, DisplayName,  
State, StartMode, PathName
```

Retrieves detailed information about services, including the executable path and start mode using CIM/WMI.

Collect Scheduled Task Artifacts:

```
Get-ScheduledTask | Select-Object TaskName, TaskPath, State
```

Lists all scheduled tasks with their names, folder paths, and current state (ready, running, disabled).

```
Get-ScheduledTask | Where-Object {$_.State -ne "Disabled"} |  
Get-ScheduledTaskInfo
```

Retrieves all enabled scheduled tasks and then queries detailed runtime information about each task.

```
Get-ScheduledTask | Where-Object {($_.State -ne 'Disabled') -and ((($_.LastRunTime  
-eq $null) -or ($_.LastRunTime -gt (Get-Date).AddDays(-7)))} | ConvertTo-Csv  
-NoTypeInfo
```

Lists all non-disabled scheduled tasks that have either never run or have run within the last seven days and outputs the results in CSV format.

Collect Event Log Artifacts:

```
Get-WinEvent -ListLog *
```

Retrieves a listing of all event logs available on the system.

```
Get-WinEvent -FilterHashtable @{LogName="Security"; ID=4625}
```

Queries the **Security** event log for Event ID **4625**, which records failed user logon attempts. The **LogName** and **ID** keys can easily be changed to interrogate other log or event IDs.

```
Get-WinEvent -FilterHashtable @{LogName="Security";  
ID=4720,4722,4724,4738,4732,1102}
```

Retrieves **Security** log events for key user and group management activities and audit log clearing.

Collect File System and Software Artifacts:

```
Get-ChildItem C:\ -Recurse -Filter calc.exe -ErrorAction SilentlyContinue |  
Select-Object -ExpandProperty FullName
```

Recursively search the `C:\` drive for files named `calc.exe` and output their full paths.

```
Get-ChildItem "$env:APPDATA\Microsoft\Windows\Recent"
```

Lists recent shortcut (LNK) files indicating accessed documents or executables.

```
Get-ChildItem C:\Windows\Prefetch
```

Retrieves a listing of Prefetch files that provide a history of recently executed applications.

```
Get-ItemProperty -Path "HKCU:\Software\Microsoft\Internet Explorer\TypedURLs"
```

Fetches user-typed URLs stored in the registry for Internet Explorer. Browser history database files for Edge, Chrome, and Firefox generally require parsing outside PowerShell, such as:

- `C:\Users\%Username%\AppData\Roaming\Mozilla\Firefox\Profiles\`
- `C:\Users\%Username%\AppData\Local\*\*\User Data\*`

```
Get-CimInstance Win32_Product | Select-Object Name, Version, InstallDate
```

Lists installed software as reported by the Windows Installer database. Querying the Windows Installer service can be slow, so faster registry-based alternatives are available, such as:

```
# List installed software from the 64-bit uninstall registry key  
Get-ItemProperty -Path  
"HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*" | Select-Object  
DisplayName, DisplayVersion, InstallDate
```

```
# List installed software from the 32-bit uninstall registry key
Get-ItemProperty -Path
"HKLM:\Software\Wow6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*" |
Select-Object DisplayName, DisplayVersion, InstallDate

# List installed software for the current user
Get-ItemProperty -Path
"HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*" | Select-Object
DisplayName, DisplayVersion, InstallDate
```

```
Get-CimInstance Win32_ShadowCopy
```

Retrieves information about the Volume Shadow Copies (snapshots) on the system.

Collect Microsoft Defender Exclusions:

```
Get-MpPreference | Select-Object -Property ExclusionPath, ExclusionExtension,
ExclusionIpAddress, ExclusionProcess
```

Retrieves all Defender antivirus exclusion settings (paths, extensions, IPs, and processes).

Collect Installed and Loaded Drivers:

```
Get-CimInstance Win32_SystemDriver | Select-Object Name, State, PathName
```

Lists all system drivers showing their service name, current state, and the full path to the driver executable.

PowerShell Incident Response Frameworks

Every incident can unfold differently, making it hard to form a single, all-encompassing playbook. Fortunately, many talented researchers have developed PowerShell-based IR and forensic frameworks to speed up evidence collection and triage, while maintaining flexibility. These frameworks provide prebuilt modules and scripts you can adapt to your investigation. Some popular examples include:

- **Kansa:** A modular incident response framework for large-scale data collection.
 - <https://soc201.com/YygzAJ>
 - **Invoke-IR (PowerForensics):** An all inclusive framework for hard drive forensic analysis.
 - <https://soc201.com/u3cBSA>
 - **PowerShell Rapid Response:** A set of WMI scripts for investigators and forensic analysts.
 - <https://soc201.com/U4iCFN>
 - **PSRecon:** Automates live host triage and outputs results in a report format.
 - <https://soc201.com/jLNgnU>
-

References

- Nair, S. (2017). *Live Response with PowerShell*. GIAC GCFA Gold Certification Paper. Retrieved from <https://www.giac.org/paper/gcfa/3393/live-response-powershell/108766>
- The PowerShell Podcast. (2023, January 30). *Using PowerShell for incident response (Episode 21)* [Video]. YouTube. <https://www.youtube.com/watch?v=nUHBkf0bLrI>
- Hull, D. (2014). *Kansa: A modular PowerShell-based Incident Response framework*. Retrieved from <https://github.com/davehull/Kansa>
- Atkinson, J. (2016). *PowerForensics: A PowerShell digital forensics framework*. Retrieved from <https://github.com/Invoke-IR/PowerForensics>
- Tomlinson, F. (2016). *PowerShell Rapid Response: A set of WMI scripts for investigators and forensic analysts*. Retrieved from <https://github.com/WiredPulse/PoSh-R2>
- Foss, G. (2015). *PSRecon: Automated data collection for live incident response*. Retrieved from <https://github.com/gfoss/PSRecon>